

Ampliación Clases Abstractas:

- **Conceptos que se incluyeron en el ejemplo:**

- 1- Clases Abstractas.
- 2- Herencia.
- 3- Sobreescritura de métodos (override).
- 4- Uso de clases en un formulario de Windows Forms.

- **Explicación bloque por bloque del ejemplo de la Unidad 9 del Cuadernillo:**

<code>namespace HolaMundo</code>	Define un contenedor lógico (proyecto) para las clases.
<code>public partial class Form1 : Form</code>	Form1 es el formulario principal del programa. Hereda de la clase Form, que es parte de la librería de Windows Forms (System.Windows.Forms). partial indica que esta clase puede estar dividida en varios archivos. El archivo Form1.Designer.cs contiene el diseño visual del formulario (por ejemplo, el Label).
<pre>public Form1() { InitializeComponent(); }</pre>	Este es el constructor de Form1. Se ejecuta al crear una instancia del formulario. InitializeComponent() configura todos los componentes gráficos (como botones, etiquetas, etc.) del formulario. Este método es generado automáticamente por el diseñador de Visual Studio.
<pre>private void Form1_Load(object sender, EventArgs e) { Robot robotin = new Robot(); label1.Text = robotin.Saludar(); }</pre>	Este método se ejecuta automáticamente cuando el formulario se carga. Se crea una instancia de la clase Robot llamada robotin. Se llama al método Saludar() de robotin y el resultado se muestra en label1 (una etiqueta en el formulario).
<pre>public abstract class Saludo { public abstract string Saludar(); }</pre>	Una clase abstracta no se puede instanciar directamente. Sirve como base para otras clases. Define un método abstracto Saludar(), que debe ser implementado por cualquier clase que herede de Saludo.
<pre>public class Robot : Saludo { public override string Saludar() { return "HOLA MUNDO!!!"; } }</pre>	Robot hereda de la clase abstracta Saludo. Usa la palabra clave override para sobrescribir el método abstracto Saludar() y proporciona su propia implementación. Retorna el mensaje "HOLA MUNDO!!!".

- **Analogía del ejemplo:**

- a- Imaginen que la clase "Saludo" es como un contrato que dice: "toda clase que me herede tiene que saber saludar".
- b- Pero no dice cómo saludar, eso lo decide cada clase hija.
- c- En el ejemplo, la clase Robot cumple con el contrato y dice: "yo saludo diciendo HOLA MUNDO!!!".

- **Vamos a ampliar el código agregando otras clases:**

- 1- Crear otras clases que hereden de Saludo, como Humano y Perro, y que saluden de forma diferente.
- 2- Cambien el código para mostrar esos saludos en el formulario.
- 3- Usen List<Saludo> para practicar polimorfismo (manejar varios objetos con un mismo tipo base).

Sería más o menos así:

- a- Agregando las nuevas clases que heredan de "Saludo"

```
public class Humano : Saludo
{
    public override string Saludar()
    {
        return "Hola, ¿cómo estás?";
    }
}

public class Perro : Saludo
{
    public override string Saludar()
    {
        return "Guau guau!";
    }
}
```

- b- Cambiar el método Form1_Load para usar varias clases:

```
private void Form1_Load(object sender, EventArgs e)
{
    // Lista de saludadores
    List<Saludo> saludadores = new List<Saludo>
    {
        new Robot(),
        new Humano(),
        new Perro()
    };

    // Mostrar todos los saludos en el Label
    string saludos = "";
    foreach (Saludo s in saludadores)
    {
        saludos += s.Saludar() + Environment.NewLine;
    }

    label1.Text = saludos;
}
```

- **Como se explican los cambios añadidos?:**

- 1- Creamos una lista de tipo Saludo, pero le agregamos objetos de diferentes clases (Robot, Humano, Perro).
- 2- Esto es un ejemplo de **polimorfismo**: todos los objetos se tratan como tipo Saludo, pero cada uno ejecuta su propia versión de Saludar().
- 3- Se construye una cadena de texto con todos los saludos y se muestra en la etiqueta label1.

=====

- **Primer Ejercicio propuesto:**

- 1- Agreguen una clase **Gato** que diga "Miau".
- 2- Usen **ComboBox** para seleccionar qué tipo de saludo quieren mostrar.
- 3- Que sea usando clases globales.

- **Segundo Ejercicio propuesto:**

- 1- Agregar una nueva clase Alien que diga "Saludos terrícolas!".
- 2- Mostrar el saludo en un MessageBox además del Label.

- **Ejercicio desafío:**

Agregar una imagen que cambie según el tipo de ser seleccionado (usando el objeto PictureBox).

=====

- **Respuestas a algunas posibles preguntas:**

¿Por qué usamos una clase abstracta en lugar de una clase normal?

Usamos una clase abstracta cuando queremos definir una **estructura común** para varias clases, pero sin dar detalles de cómo funcionan sus métodos.

Es como decir:

"Todos los seres vivos pueden saludar, pero *cada uno lo hace a su manera*."

Una clase abstracta:

- No se puede instanciar directamente (no podés hacer new SerVivo()).
- Obliga a las clases hijas a **implementar ciertos métodos**, como Saludar() en nuestro caso.
- Sirve como "molde" para organizar el código y que todas las clases hijas cumplan con el mismo contrato.

Ejemplo de la vida real:

Es como decir que todos los vehículos tienen que tener una forma de moverse (Mover()), pero no todos se mueven igual. Una bicicleta, un auto y un avión lo hacen distinto, pero todos tienen que implementar esa acción.

¿Qué ventaja tiene usar una lista de SerVivo en vez de hacer todo con if?

Usar una lista de SerVivo y aprovechar el **polimorfismo** te permite trabajar con diferentes tipos de objetos **de forma más flexible y organizada**.

Si usáramos solo if o switch, tendríamos que verificar manualmente cada tipo de objeto y escribir mucho código repetido.

Con polimorfismo, el código queda limpio y escalable:

```
foreach (SerVivo ser in lista)
{
    ser.Saludar(); // Cada objeto sabe cómo saludar, sin importar de qué tipo es
}
```

Ventajas concretas:

- Podés agregar nuevos tipos (como Alien) sin cambiar el código del foreach.
- Mantenés el código ordenado, fácil de leer y mantener.
- Aprovechás la orientación a objetos en su máximo potencial.

¿Qué pasaría si no usamos override?

Si no usamos override cuando heredamos un método abstracto, el código no compila. El compilador te va a marcar un error que dice algo como:

"La clase 'Humano' no implementa el miembro abstracto 'Saludar' de 'SerVivo'."

Esto pasa porque el método Saludar() fue declarado como abstracto, y eso significa que todas las clases que heredan de SerVivo están obligadas a implementarlo.

¿Qué significa override?: Es una palabra clave que indica que estamos reescribiendo un método que viene de una clase base.

Extra: ¿Y si Saludar() no fuese abstracto?

Si lo definieras así no sería abstracto:

```
public virtual string Saludar()  
{  
    return "Saludos genéricos.";  
}
```

Entonces las clases hijas podrían elegir si lo sobrescriben o no. Pero si es abstract, es obligatorio.

• Resumen final de conceptos:

Concepto	¿Qué significa?	Ejemplo
abstract	Es un modelo, no tiene implementación propia.	abstract class SerVivo.
override	Es la implementación concreta en la clase hija.	public override string Saludar().
polimorfismo	Muchos tipos diferentes comparten una interfaz común.	Lista de SerVivo, cada uno saluda distinto.